

ML Major 2

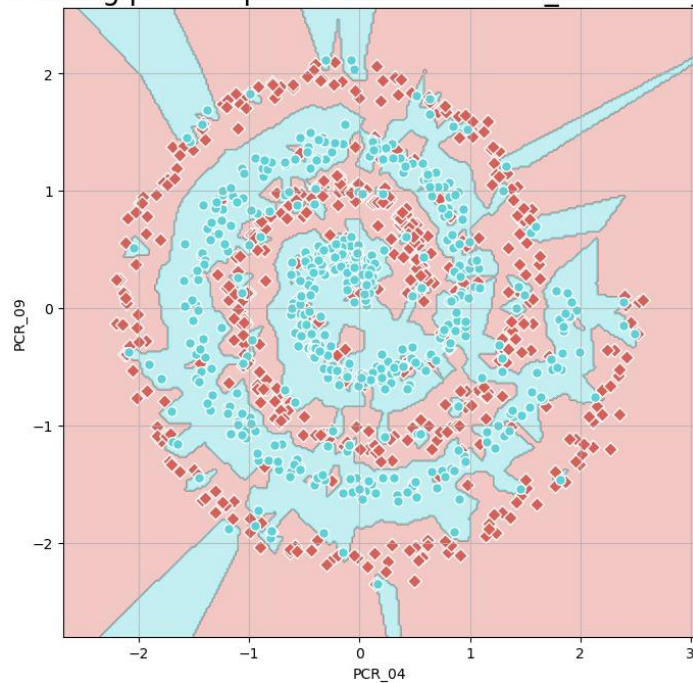
Submitters:

Husain Azaqy: 213382104

Galor Lazar: 325028876

(Q1) Train a k-NN model using $k = 1$ on your training set and use the `visualize_clf` method to visualize the resulted decision regions (with appropriate title and labels).

Training plot of spread relative to PCR_04 & PCR_09



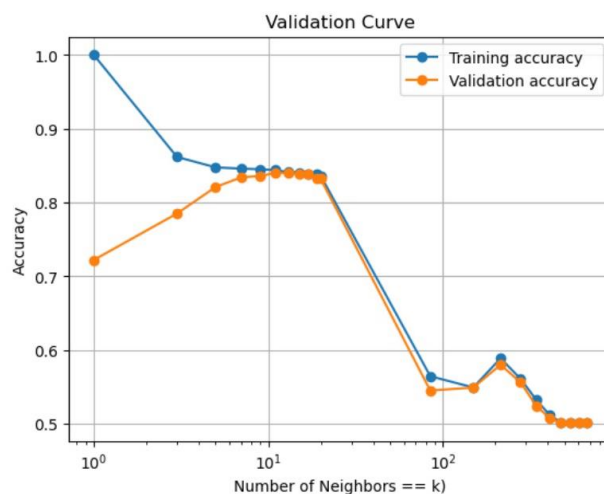
(Q2) Answer: Which k is the best? What are its average training and validation accuracies (estimated by cross-val.)? Which k values cause overfitting and underfitting and why?

We choose the best k by finding the k that provides the best validation accuracy because the validation error tells us how good a model generalizes .

The best k for predicting the spread class using PCR_04 and PCR_09 is k=11 with average training and validation accuracy both being 84%.

We know that overfitting is when the model performs very well on the training data but not well on the validation because the model is complex and modified too well on the training data and the noise it includes. We can see that in k =1 where the Training accuracy is 100% but the validation data accuracy is 72.2%.

Underfitting occurs when the model tries to be too general and tries to find too many neighbors that may be far away from the original point we are trying to classify. By this, the model gets in both training and validation datasets a very low accuracy. We can see that in the k's that are greater than 85, they have both very low training and validation datasets accuracy.

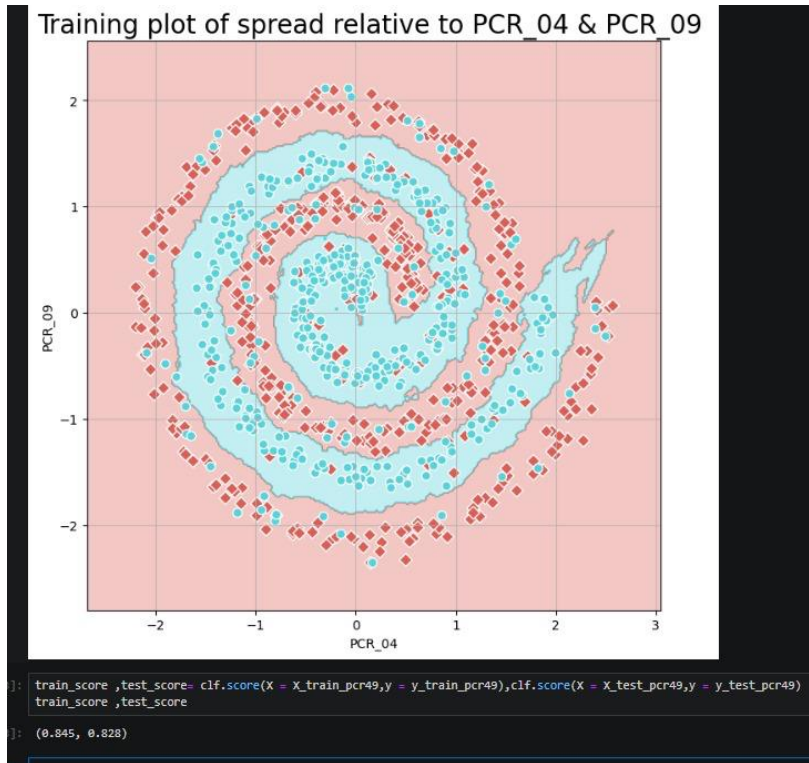


```
k: 1, Training Accuracy: 1.0000, Validation Accuracy: 0.7220
k: 3, Training Accuracy: 0.8617, Validation Accuracy: 0.7850
k: 5, Training Accuracy: 0.8476, Validation Accuracy: 0.8210
k: 7, Training Accuracy: 0.8459, Validation Accuracy: 0.8340
k: 9, Training Accuracy: 0.8450, Validation Accuracy: 0.8360
k: 11, Training Accuracy: 0.8439, Validation Accuracy: 0.8400 <-- highest validation accuracy
k: 13, Training Accuracy: 0.8411, Validation Accuracy: 0.8400
k: 15, Training Accuracy: 0.8407, Validation Accuracy: 0.8380
k: 17, Training Accuracy: 0.8393, Validation Accuracy: 0.8380
k: 19, Training Accuracy: 0.8381, Validation Accuracy: 0.8330
k: 20, Training Accuracy: 0.8363, Validation Accuracy: 0.8320
k: 85, Training Accuracy: 0.5644, Validation Accuracy: 0.5450
k: 150, Training Accuracy: 0.5491, Validation Accuracy: 0.5490
k: 215, Training Accuracy: 0.5887, Validation Accuracy: 0.5800
k: 280, Training Accuracy: 0.5609, Validation Accuracy: 0.5560
k: 345, Training Accuracy: 0.5323, Validation Accuracy: 0.5240
k: 410, Training Accuracy: 0.5121, Validation Accuracy: 0.5080
k: 475, Training Accuracy: 0.5019, Validation Accuracy: 0.5010
k: 540, Training Accuracy: 0.5010, Validation Accuracy: 0.5010
k: 605, Training Accuracy: 0.5010, Validation Accuracy: 0.5010
k: 670, Training Accuracy: 0.5010, Validation Accuracy: 0.5010
```

The value of k with the highest validation accuracy is: 11

(Q3) Use the optimal k value you found and retrain a k-NN model on all the training samples. In your report: plot the decision regions of this final model (using visualize_clf) and write its test accuracy (computed on the separate test split) of this model.

For $k=11$:



Training accuracy score: 84.5%

Test accuracy score: 82.8%

(Q4) Compare the boundaries of the two models you have trained in (Q1) and (Q3). Discuss the results and the exhibited behaviors (2-3 sentences).

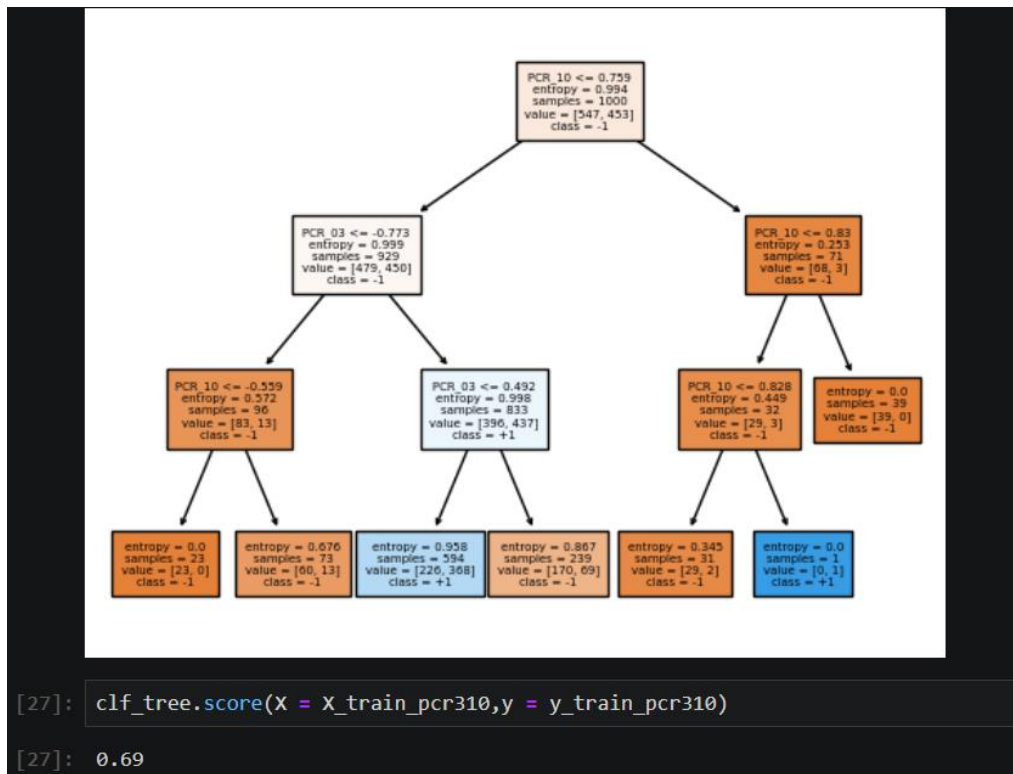
Comparison between $k=1$ to $k=11$:

As we can clearly see, the $k=11$ graph is much smoother and less sensitive for outliers, which gives us much better performance and generalization.

Unlike the $k=1$, the $k=1$ model uses only one neighbor, in its graph we can see many different small areas of color changing around single points that may count as outliers in the real dataset. That happens because this time the model uses only the 1 nearest neighbor for the predictions and doesn't generalize it by checking the data and the area that the point itself found in.

(Q5) Train a model with ID3 and max_depth=3 (not including the root level; use the entire training set, i.e., all the features after preprocessing from all the training samples). What is the training accuracy? Visualize the trained tree using plot_tree (provide feature and class names; use filled=True) and attach the plot to your report. The plot should be readable!

The training accuracy is 69 %



(Q6) Using 5-fold cross-validation, tune the two hyperparameters by performing a grid search (see GridSearchCV). Find the combination yielding the best validation error for predicting the risk class. You should:

a. Choose appropriate ranges for both hyperparameters. This may require a few attempts. To make things quicker when trying to find appropriate hyperparameter ranges, you can start by using only 2 folds.

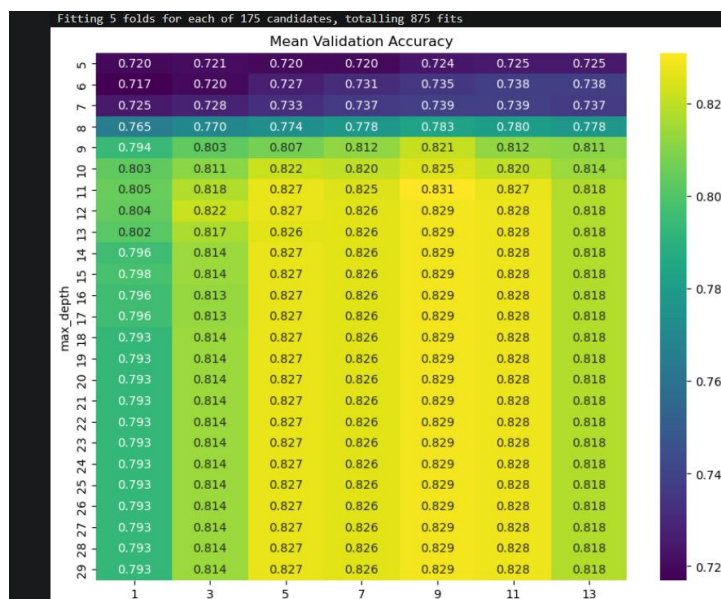
Our ranges:

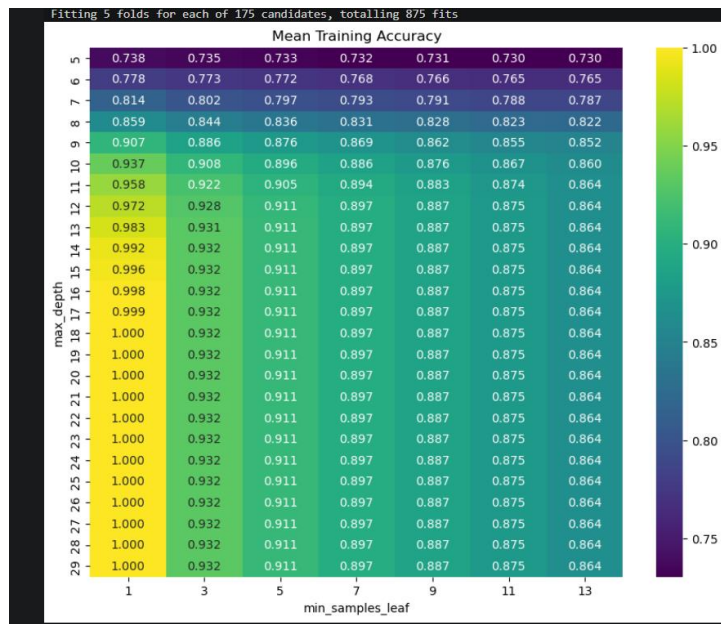
- “max_depth” range = [5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29]
- “min_samples_leaf” range: [1, 3, 5, 7, 9, 11, 13]

b. Since we tune two hyperparameters, instead of a validation curve, plot two heatmaps (seaborn / pyplot), for each of the cross-validated training and validation accuracy (the heatmaps should roughly look like the ones to the right). Plot the appropriate “ticks” on both axes and use annotations (annot=True) to explicitly write accuracies inside heatmap cells. Important: The plots should be readable and informative!

c. Add the 2 plots to your report and specify which hyperparameter combination is optimal.

The heat maps:





The optimal hyper parameters are:

'max_depth': 18

'min_samples_leaf': 1

Yielding 88.3% training and 83.1% validation mean accuracy.

d. Write a hyperparameter-combination that causes underfitting.

Answer: consider

'max_depth': 5

'min_samples_leaf': 13

Yielding 73.0% training and 72.5% validation mean accuracy.

e. Write a hyperparameter-combination that causes overfitting.

Answer: consider

'max_depth': 18

'min_samples_leaf': 1

Yielding 100.0% training and 79.3% validation mean accuracy.

f. Add a short discussion regarding why each specific hyperparameter-combination from sub-questions 'd' and 'e' resulted in under/over-fitting.

Answer:

- Underfitting example: (d)
 - Low depth restricts the model to a low number of separation regions.
 - High `min_samples_leaf` leads to the model trying to generalize too much.
- Overfitting example: (e)
 - High depth gives the model a too high ability to separate the datasets to many regions.
 - Low `min_sample_leafs` leads to the model not trying to generalize much (here `min_sample_leafs == 1`, meaning almost no generalization in this example)

Both result in underfitting.

Both result in overfitting.

Q7) Write the number of hyperparameter combinations that were evaluated in your grid search. Had you wished to tune a third hyperparameter, how would that affect the number of combinations? Shortly discuss how searching over additional hyperparameters affects the total number of possible combinations.

Answer:

```
Fitting 5 folds for each of 175 candidates, totalling 875 fits
```

Given a third param with a grid range of size k , will have a total of $175 \cdot k$ candidates and $875 \cdot k$ fits.

(Q8) Use the optimal hyperparameter combination you found and retrain a decision tree on all the training samples. In your report write the test accuracy of this model.

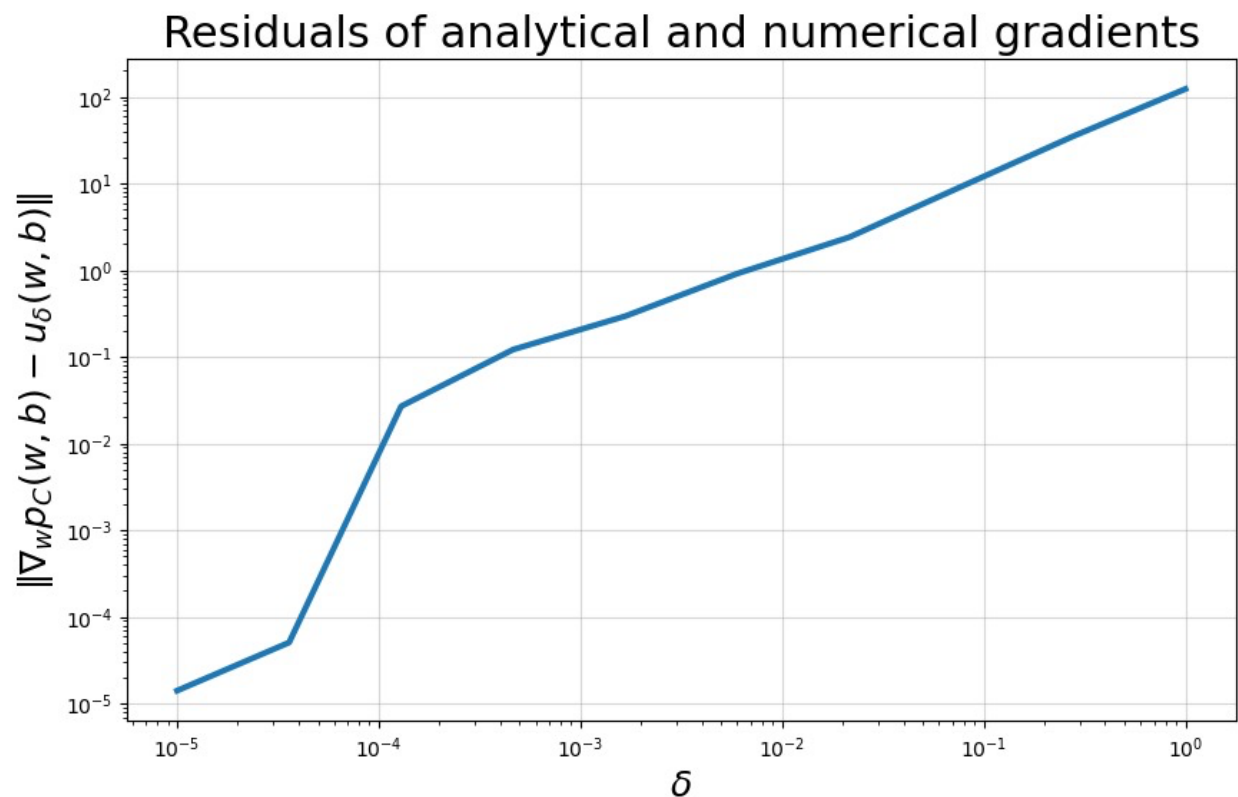
Answer: The models test accuracy 82.0%

(Q9) Using PCR_04, PCR_09, generate a plot that compares the numerical gradients to the analytic gradients. Do this by running the following command: Attach the plot to your report. Briefly discuss and justify the demonstrated behavior.

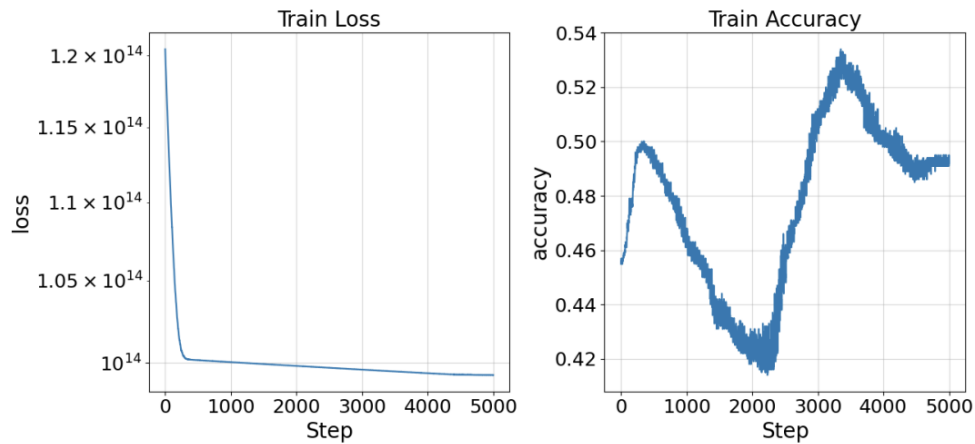
`compare_gradients(X_train, y_train, deltas=np.logspace(-5, 0, 10`

We know that as soon as the delta goes to zero, the numerical gradient approaches the analytic gradient. If so, as the delta is smaller, we can expect that the differences between the analytic and the numerical to be smaller.

Indeed, as we can see that happens on the graph. In contrast, as well as we increase the delta we get bigger differences between the two gradients.

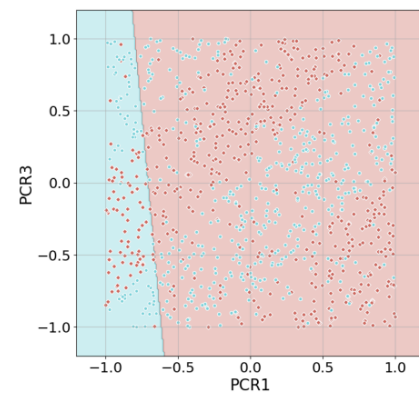


(Q10) We trained a SoftSVM with $c = 1e11$, $lr = 2e-14$ and obtained the following curves:



The resulting model induces the following decision regions (to the right).

Discuss the behavior of the loss and accuracy during the training of the model and any interactions you observe between the loss and the accuracy. Does this interaction match your expectations? If it does – explain why. If it does not – try to settle it.



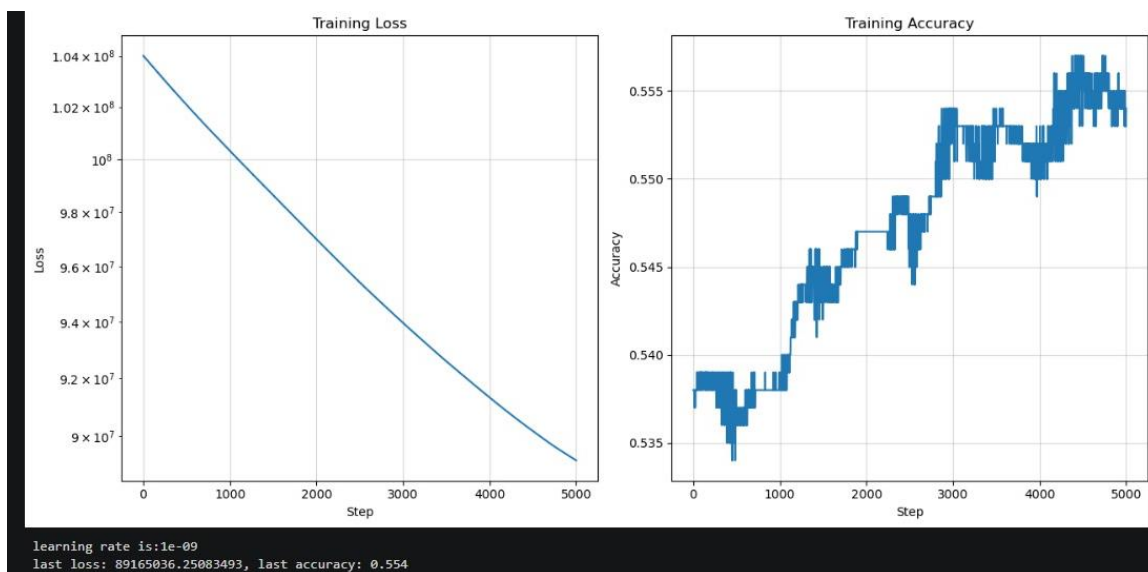
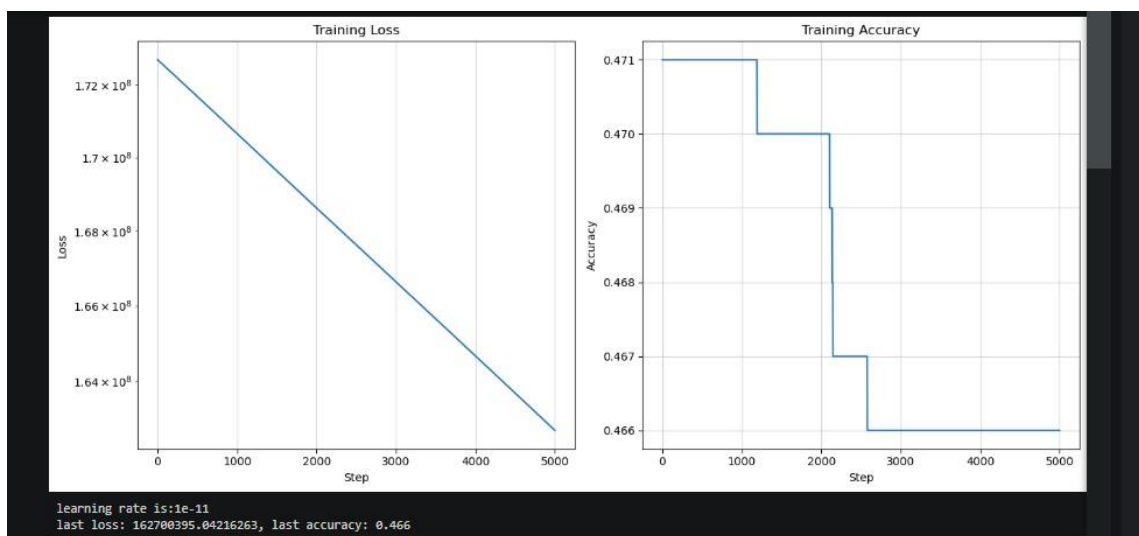
The learning rate here is very low, so we know we should get that the convergence into the solution will be very, but when the iterations $\rightarrow$ infinity, it will converge. As we can see in the graph, because of the small learning rate, by the accuracy graph we can see that even after 5000 steps the accuracy converged to a poor one and the loss is still high.

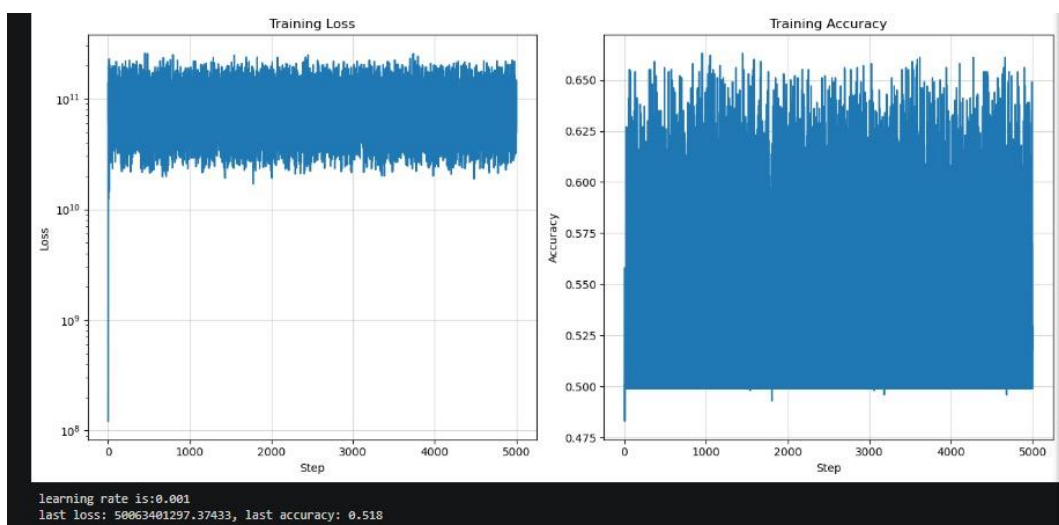
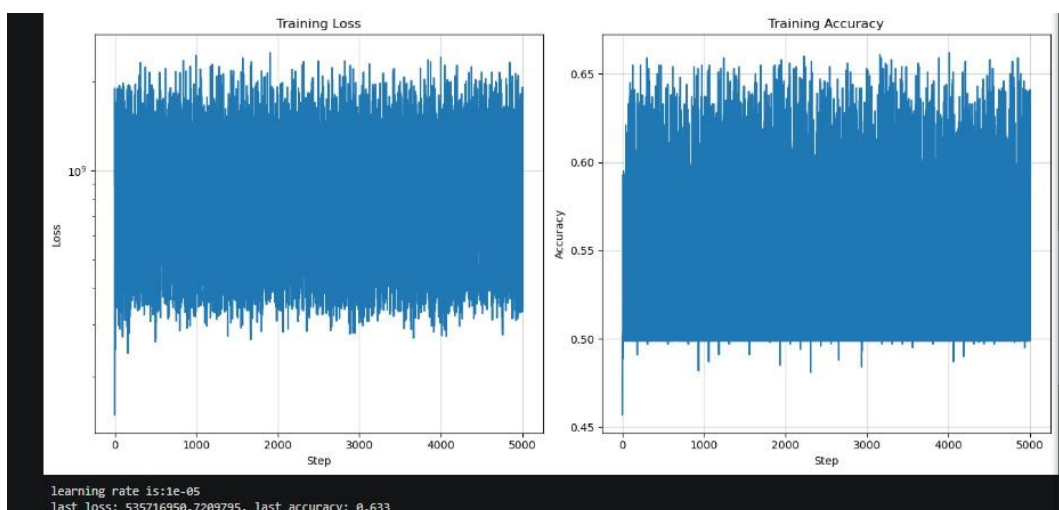
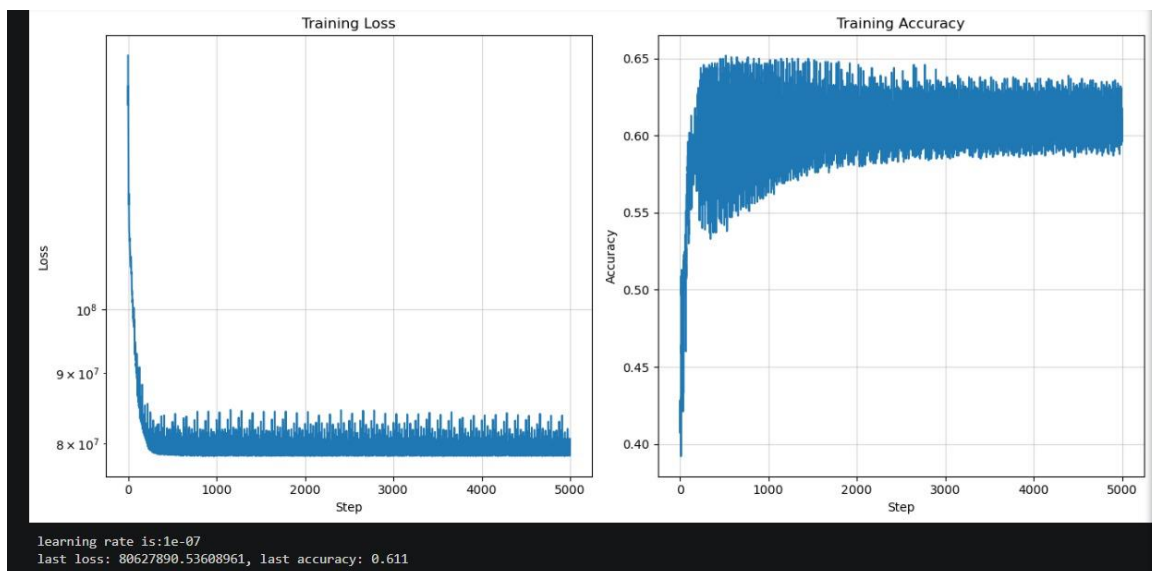
Moreover, because C is very large, there is a significant **punishment** for a wrong prediction and it tries to avoid them as much as it can. By that, the model behaves like a Hard-SVM model more than Soft-SVM. As we can see in the graph, the data isn't linear separable, so the model doesn't succeed in getting good accuracies on predictions and the accuracy doesn't just get better as we get more iterations.

(Q11) Add the plots to the report and explain which learning rate you would choose and why.

We would choose learning rate = 1×10^{-7} (third plot). We can easily see that in comparison to other models, we get that this model gets the highest accuracy, and gets very low loss in fewer steps. Also as we can see in the graph, as we continue with the steps, the accuracy remains around 61%, which means that we probably got to the global minimum. Also, its loss is lower than all other models we saw. So we chose that model because it was better considering our two evaluation metrics.

Note that the graph in plot 3 is less “noisy” in the sense that the noise/jumps somewhat converge.



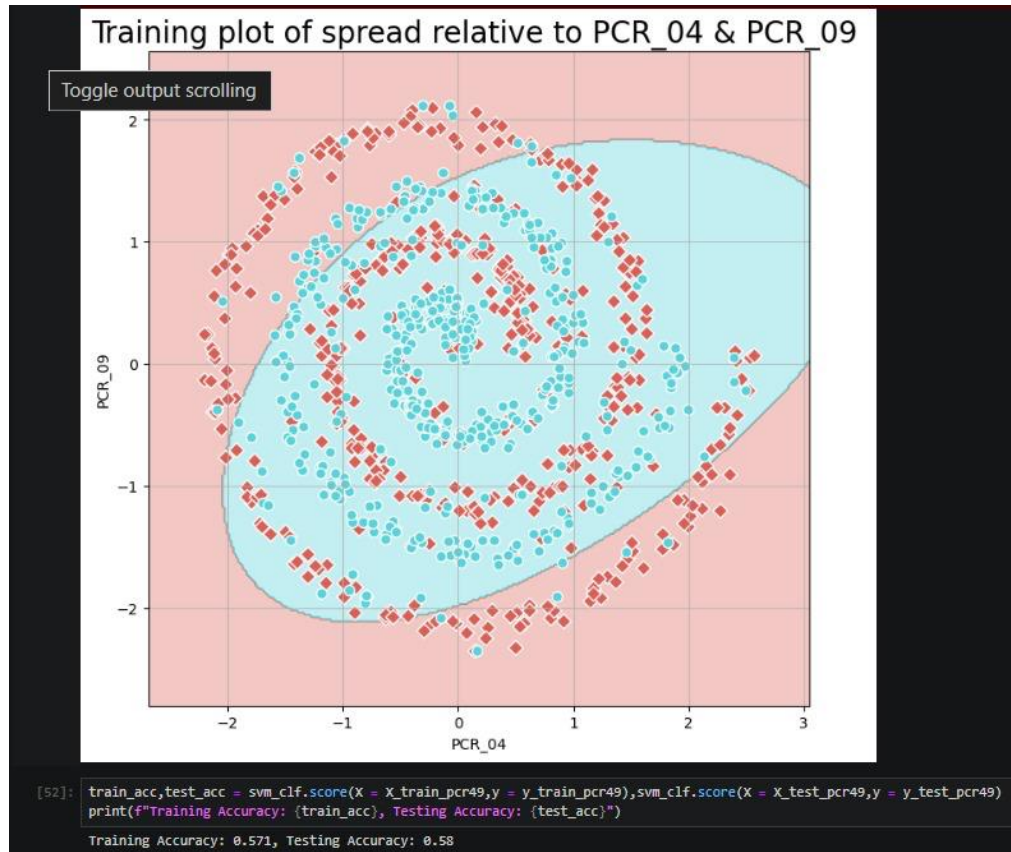


(Q12) In your report:

- Plot the trained model's decision regions.
- Write the respective training and test accuracies of the model.

Training Accuracy: 57.1%

Testing Accuracy: 58%



(Q13) For the sake of simplicity, in the following question, we will introduce and use the Laplacian kernels:

$$K(\mathbf{u}, \mathbf{v}) = \exp\{-\gamma \|\mathbf{u} - \mathbf{v}\|_2\}$$

Given a sample set $S = \{(x_i, y_i)\}_{i=1}^m$ where $x_i \in D, y_i \in \{-1, 1\}$, and $|\{x_i: y_i = 1\}| = \frac{m}{2}$:

- a. Dual Solution and Prediction Rule: Given the dual solution α such that $\forall i \in [m], \alpha_i = 1$, write the prediction rule $h(x)$ using the Laplacian kernels.

Now assume that there exist $\delta > 1$ that for every distinct $i, j \in [m]$, we have:

$$\|x_i - x_j\|_2 > 3\delta$$

Additionally, for every sample $x \in D$ there exists a tuple $(x_i, y_i) \in S$ such that:

$$y = y_i \text{ and } \|x - x_i\|_2 < \delta - 1$$

- b. Take the prediction rule from before and show the transition to the following form:

$$h(x) = \text{sign} \left(\sum_{j \in \{i \in [m]: y_i = 1\}} K(x, x_j) - \sum_{j \in \{i \in [m]: y_i = -1\}} K(x, x_j) \right)$$

Given a new sample x with classification $y = 1$

- c. Prove the following inequality:

$$\sum_{j \in \{i \in [m]: y_i = 1\}} K(x, x_j) > \exp\{-\gamma(\delta - 1)\}$$

- d. Prove Another Inequality:

$$\sum_{j \in \{i \in [m]: y_i = -1\}} K(x, x_j) \leq \frac{m}{2} \exp\{-\gamma(2\delta - 1)\}$$

Hint: You can prove this using the norm triangle inequality or geometric arguments.

- e. Show the Prediction Result: Given $\gamma = \ln\left(\frac{m}{2}\right)$, show that $h(x) = 1$.
- f. **Prove the General Prediction Rule:** Based on the previous proofs, show that the prediction rule $h(x)$ is always correct under the given assumptions.

a)

Noting that $K(x, x_i) = \exp\{-\gamma \|x - x_i\|_2\}$

$$\begin{aligned} h(x) &= \text{sign} \left(\sum_{i \in [m], \alpha_i > 0} (\alpha_i y_i K(x, x_i)) \right) = \text{sign} \left(\sum_{i \in [m]} (y_i K(x, x_i)) \right) \\ &= \text{sign} \left(\sum_{i \in [m]} (y_i \exp\{-\gamma \|x - x_i\|_2\}) \right) \end{aligned}$$

b) Observe:

$$\begin{aligned}
 \text{sign} \left(\sum_{i \in [m]} (y_i K(x, x_i)) \right) \\
 &= \text{sign} \left(\sum_{j \in \{i \in [m], y_i = 1\}} (y_i K(x, x_j)) + \sum_{j \in \{i \in [m], y_i = -1\}} (y_i K(x, x_j)) \right) \\
 &= \text{sign} \left(\sum_{j \in \{i \in [m], y_i = 1\}} (K(x, x_j)) - \sum_{j \in \{i \in [m], y_i = -1\}} (K(x, x_j)) \right)
 \end{aligned}$$

Note that the last equality is just plugging in the values of y_i respectively.

c) *by the piazza: $m > 2$

Given the new example x with $y=1$ we know that there exist a tuple (x_k, y_k)
s.t

$$y = y_k \text{ and } \|x - x_k\|_2 < \delta - 1$$

$$\begin{aligned}
 \sum_{j \in \{i \in [m], y_i = 1\}} (K(x, x_j)) &= \sum_{\substack{j \in \{i \in [m], y_i = 1\} \\ j \neq k}} (K(x, x_j)) + K(x_k, x_j) \\
 &= \sum_{\substack{j \in \{i \in [m], y_i = 1\} \\ j \neq k}} (K(x, x_j)) + \exp\{-\gamma \|x - x_k\|_2\} \geq \exp\{-\gamma \|x - x_k\|_2\} \\
 &\geq \exp\{-\gamma(\delta - 1)\}
 \end{aligned}$$

■

Notes:

- we can separate x_k from the sum since $y_k = 1$ too so we originally summed over x_k too.
- The $\geq$ inequality is because the sum isn't empty ($m > 2$) and K is an exponent function. Exponents is always greater than 0. Therefore a non empty sum of them is greater than 0 too. The last $\geq$ is because of the assumption about x_k and x .

d) Note that:

$$\exists x_k \text{ s.t. } \|x - x_k\|_2 < \delta - 1 \quad \forall x \in D$$

$$\exists \delta > 1 \text{ s.t. } \|x_i - x_j\|_2 > 3\delta \quad \forall i \neq j,$$

$$\begin{aligned} \text{Therefore we get for such } x_k \text{ and } y_k = 1, \quad \|x - x_j\| &= \|x - x_k + x_k - x_j\| \\ &= \|(x - x_k) - (x_j - x_k)\| \geq \|x - x_k\| - \|x_j - x_k\| \geq |3\delta - (\delta - 1)| = |2\delta - 1| \end{aligned}$$

Important: the last inequality $\geq$ works since delta is greater than one, making 3 delta and delta minus one greater than zero both.

All in all:

$$\begin{aligned} \sum_{\{i \in [m], \alpha_i > 0: y = -1\}} k(x, x_j) &= \sum_{j \in \{i \in [m], \alpha_i > 0: y = -1\}} \exp(-\gamma \|x - x_j\|_2) \\ &\leq \sum_{j \in \{i \in [m], \alpha_i > 0: y = -1\}} \exp(-\gamma(2\delta - 1)) = \frac{m}{2} \exp(-\gamma(2\delta - 1)) \end{aligned}$$

■

e) The prediction result given $\gamma = \ln\left(\frac{m}{2}\right)$ and such x labeled 1.

$$h(x) = \text{sign} \left(\sum_{j \in \{i \in [m], y_i = 1\}} (K(x, x_j)) - \sum_{j \in \{i \in [m], y_i = -1\}} (K(x, x_j)) \right)$$

$$\text{So we need to show: } \left(\sum_{j \in \{i \in [m], y_i = 1\}} (K(x, x_j)) - \sum_{j \in \{i \in [m], y_i = -1\}} (K(x, x_j)) \right) > 0$$

Using C and D, since we have:

$$\begin{aligned} &\left(\sum_{j \in \{i \in [m], y_i = 1\}} (K(x, x_j)) - \sum_{j \in \{i \in [m], y_i = -1\}} (K(x, x_j)) \right) \\ &> e^{(-\gamma(\delta-1))} - \frac{m}{2} e^{(-\gamma(2\delta-1))} \end{aligned}$$

plugging in $\gamma = \ln\left(\frac{m}{2}\right)$ we get:

$$\begin{aligned} e^{(-\gamma(\delta-1))} - \frac{m}{2} e^{(-\gamma(2\delta-1))} &= e^{\left(-\ln\left(\frac{m}{2}\right)(\delta-1)\right)} - \frac{m}{2} e^{\left(-\ln\left(\frac{m}{2}\right)(2\delta-1)\right)} = \\ e^{-\ln\left(\frac{m}{2}\right)^{\delta-1}} - \frac{m}{2} e^{\ln\left(\frac{m}{2}\right)^{2\delta-1}} &= \frac{2^{\delta-1}}{m} - \left(\frac{2}{m}\right)^{-1} \frac{2^{2\delta-1}}{m} = \frac{2^{\delta-1}}{m} - \frac{2^{2\delta-2}}{m} > 0 \end{aligned}$$

The last inequality holds since $m > 2$.

■

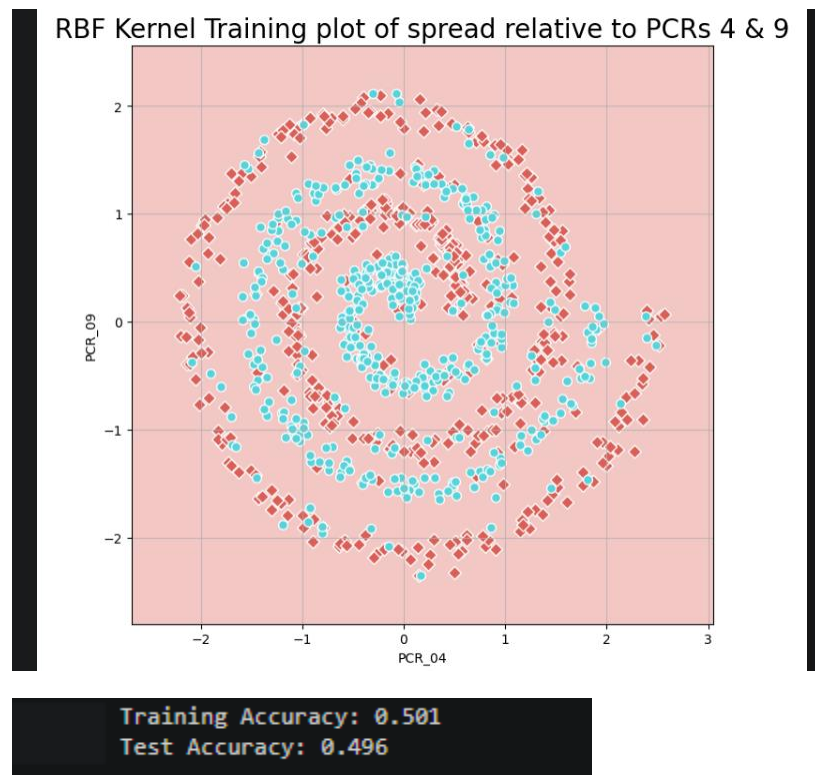
f. Note that C and D still hold assuming $y = -1$ by summing over j s.t. $y_i = -1$ in **C**

and $y_i = 1$ in **D** instead of the original summation. Therefore instead of getting in the final stage we get that the input of the sign function in $h(x)$ is negative. Yielding $h(x) = -1$ as needed. Proving that $h(x)$ is a successful prediction in any case under the given assumptions.

■

(Q14) Use `sklearn.svm.SVC` to train an SVM with an RBF kernel on the two features and the spread variable, with $C=1$ and $\gamma=1e-7$. Plot the model's decision regions and include this plot in your report. Additionally, provide the training and test accuracies of the model. Based on these accuracies, would you classify the model as underfitting, overfitting, or as a reasonable fit?

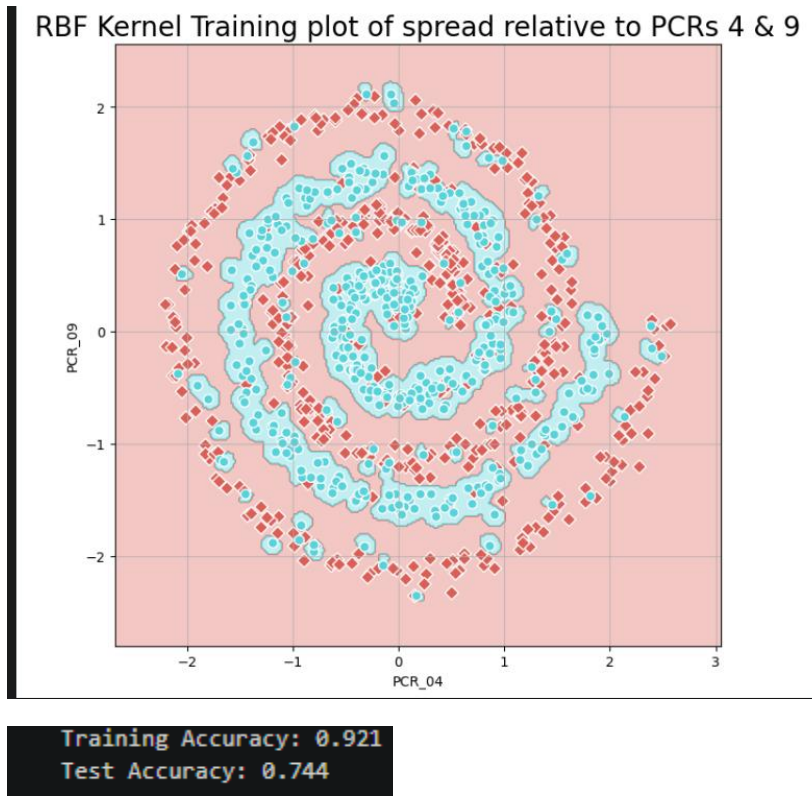
Answer:



Based on these accuracies, the model seems to classify all points as red, therefore underfitting.

(Q15) Use `sklearn.svm.SVC` to train an SVM with an RBF kernel on the two features and the spread variable, with $C=1$ and $\gamma=200$. Plot the model's decision regions and include this plot in your report. Additionally, provide the training and test accuracies of the model. Compare the decision regions of this model with the k-NN model with k optimal from (Q3). We expected the two models to be very similar, but we should still see significant differences. Try to explain the differences (there isn't a single correct answer). Hint: think what happens numerically when $\gamma = 200$ and $\|x - x_i\|_2^2$ is not very small.

Answer:



Based on the above accuracies the model is something between reasonably fitted to overfitted.

Model comparison:

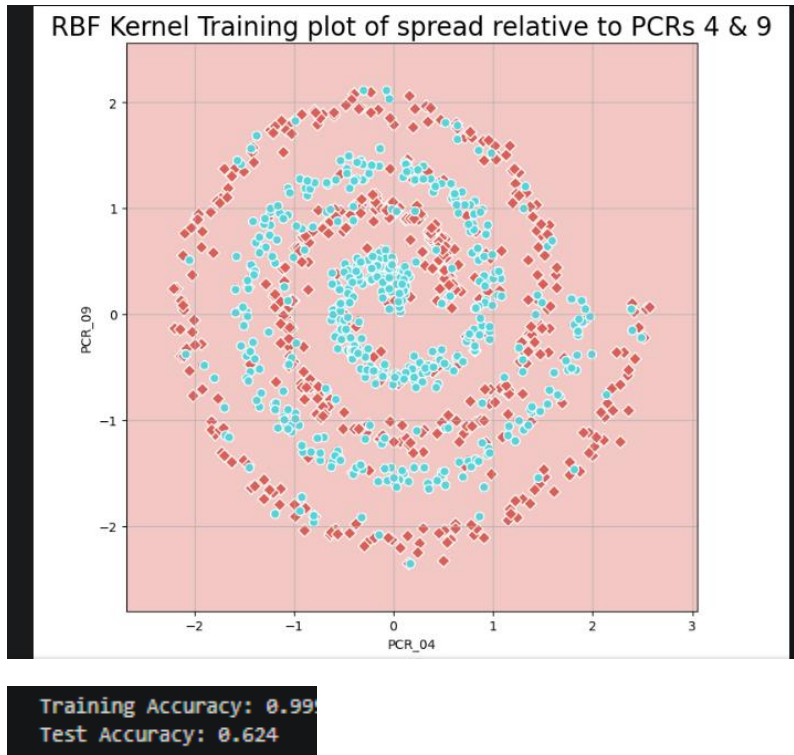
The optimal K-NN we found mimics the general “trend” of the data distribution, it does miss blue points that are surrounded by red points heavily but most of the time it is okay.

The RBF kernel model we just plotted follows a more “tailored” approach to splitting the data, resulting in very high training accuracy while maintaining somewhat high test accuracy.

Regarding the numerical aspect when $\gamma\|x - x_i\|_2^2$ is not very small, taking the negative exponent of this results in zero most of the time. Making the model behave more like a 1-NN in such case.

(Q16) Use `sklearn.svm.SVC` to train an SVM with an RBF kernel on the two features and the spread variable, with $C=1$ and $\gamma = 5000$. Plot the model's decision regions and include this plot in your report. Additionally, provide the training and test accuracies of the model. Based on these accuracies, would you classify the model as underfitting, overfitting, or as a reasonable fit?

Answer:



The model is so overfitted the blue regions are so small that they don't appear behind the blue dots icons. Effectively circling all blue points each and applying the default red (-1) classification to the rest of the space.